Module 10: Linux Exploitation and Privilege Escalation

```
# Exercise 05: Exploring GTFOBins Techniques
```

Scenario

GTFOBins Exploitation & Automated Privilege Escalation Auditing leverages curated Unix binary bypass techniques to escalate local privileges to `root`. When administrative users assign SUID bits (`u+s`) to standard command-line utilities, binaries can be manipulated via command flags, environment variables, or interactive subshells to break out of restricted execution environments.

Key technical concepts in GTFOBins enumeration & SUID auditing:
1. **LinPEAS Local Privilege Escalation Audit**: Running **./linpeas.sh** to scan system configurations, kernel exploits, cron jobs, writable paths, and SUID binaries. Highlighting suspicious non-standard SUID files under the *Interesting Files* section (**/usr/bin/pkexec**, custom suid binaries).
2. **SUID3NUM Specialized Enumeration**: Executing **python3 ./suid3num.py** from **/home/ubuntu** to automatically parse setuid/setgid binaries, cross-referencing found binaries against the GTFOBins database.
3. **GTFOBins Vector Identification**: Analyzing SUID3NUM output sections (*Interesting Stuff*, *GTFOBins list*) to identify exploitable binaries and command syntax for obtaining elevated root shells.
4. **Auto-Exploitation Testing**: Testing SUID3NUM `--auto-exploit` features, demonstrating that manual verification of GTFOBins commands remains essential for reliable privilege escalation.

In this exercise, we log into **Ubuntu** VM (**ubuntu** / **toor**), obtain root privileges (`sudo su`), execute `linpeas.sh`, identify suspicious SUID binaries, navigate to `/home/ubuntu`, execute `suid3num.py`, inspect GTFOBins references, and evaluate auto-exploitation capabilities.

Objective

The objective of this lab is to help students learn how to:
- Authenticate to Ubuntu target VM (**ubuntu** / **toor**)
- Execute automated LinPEAS audit: **chmod 755 linpeas.sh && ./linpeas.sh**
- Audit SUID binaries flagged under *Interesting Files*
- Make SUID3NUM script executable: **chmod 755 suid3num.py**
- Run SUID3NUM enumeration tool: **python3 ./suid3num.py**
- Audit *Interesting Stuff* and *GTFOBins list* sections for exploitable binaries
- Identify suspicious SUID binaries flagged by LinPEAS/SUID3NUM
- Understand manual GTFOBins breakout syntax versus automated exploitation scripts

---

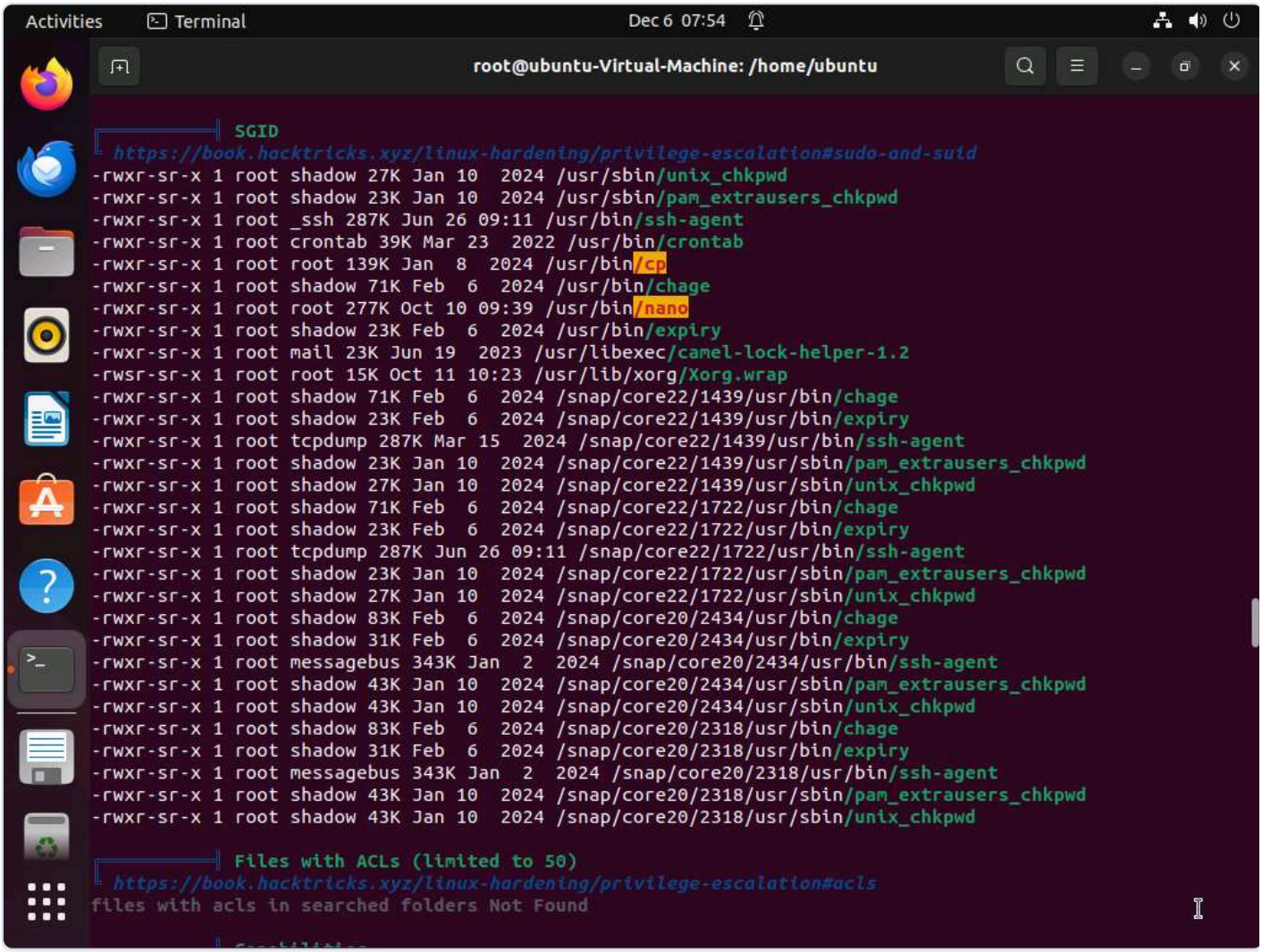
Task 1: Execute LinPEAS Audit, Enumerate SUID Files with SUID3NUM, and Analyze GTFOBins Privilege Escalation Vectors

In this task, we execute `linpeas.sh` on Ubuntu, run `suid3num.py`, review GTFOBins binary references, and identify SUID binaries.

Lab Tasks

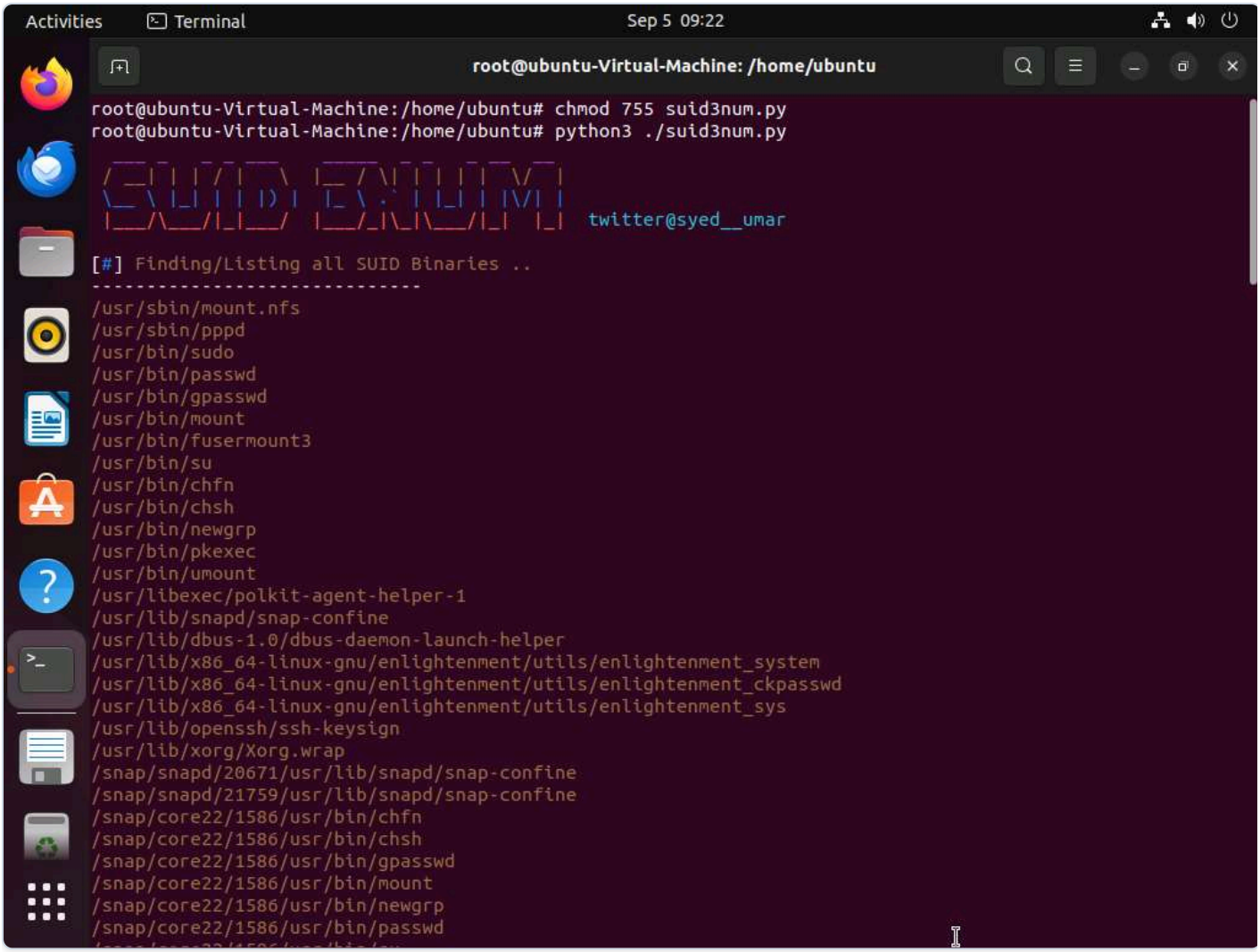
1. Log into **Ubuntu** VM using credentials (**ubuntu** / **toor**). Open terminal window and obtain root privileges:
   sudo su

2. Set permissions and execute **LinPEAS** enumeration script:
   chmod 755 linpeas.sh
   ./linpeas.sh
   Observe output: Scroll down to *Interesting Files* section to identify suspicious files with SUID bit set.

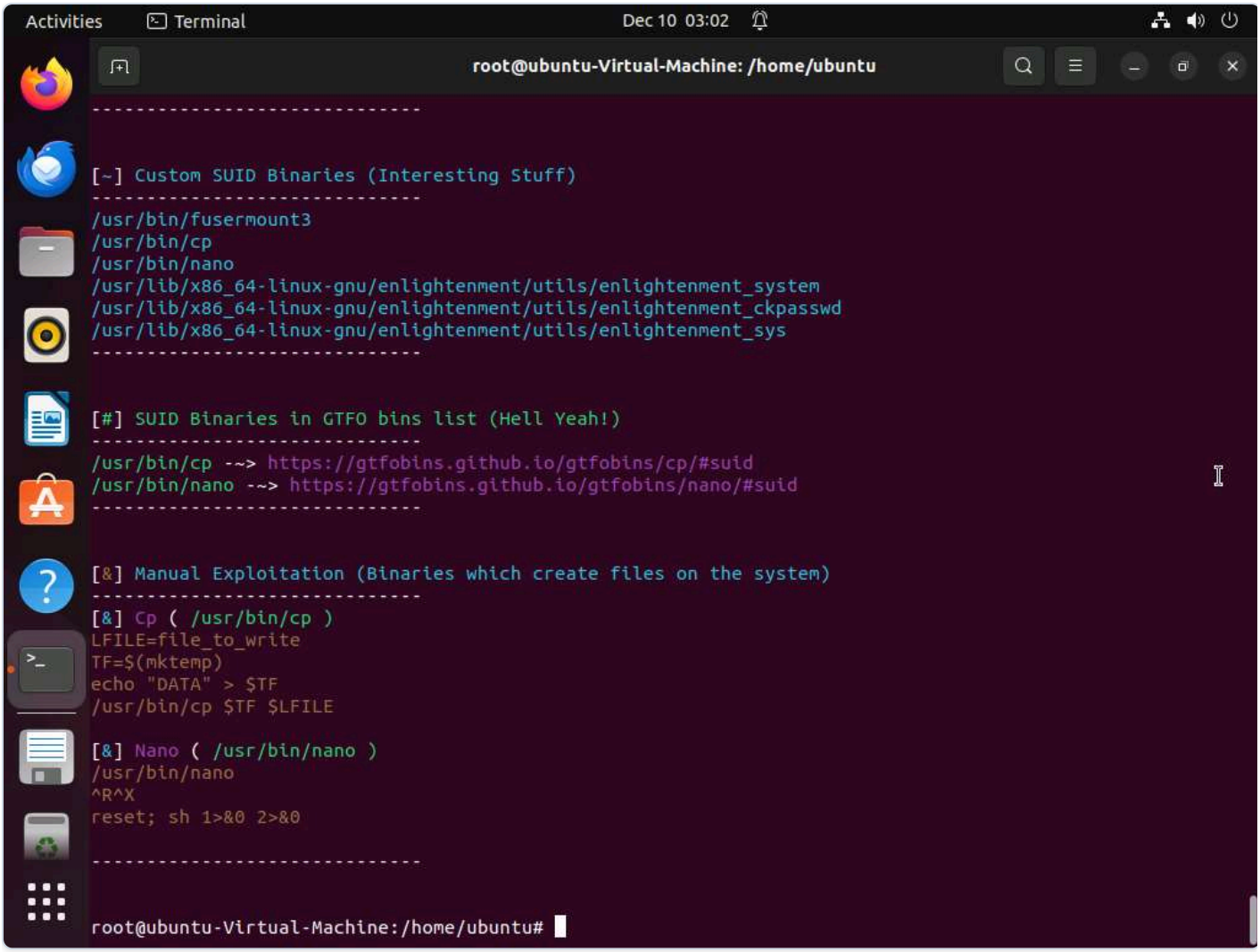


1. Locate **SUID3NUM** enumeration tool at **/home/ubuntu/suid3num.py**. Grant execution permissions:
   chmod 755 suid3num.py

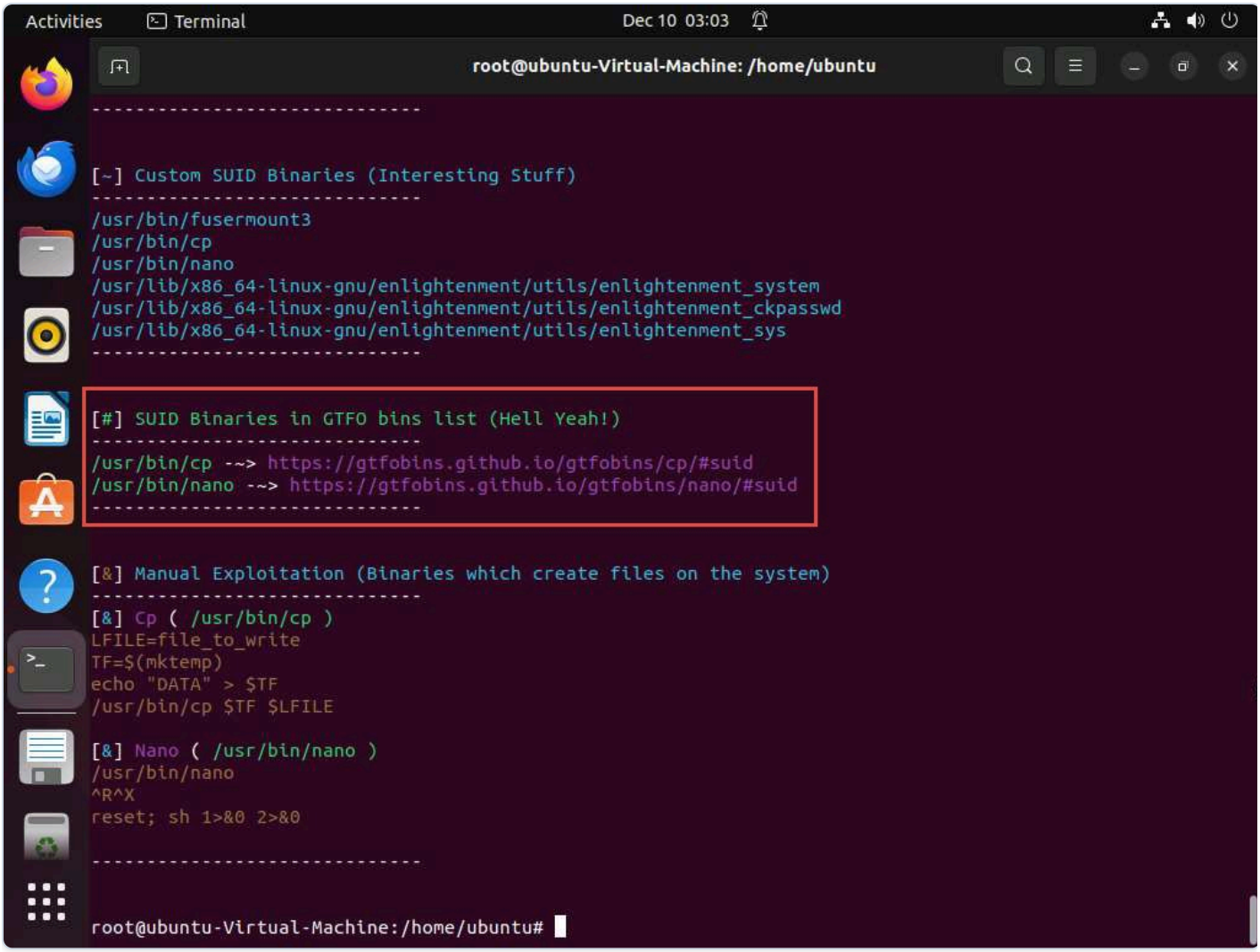
2. Execute SUID3NUM enumeration tool:
   python3 ./suid3num.py



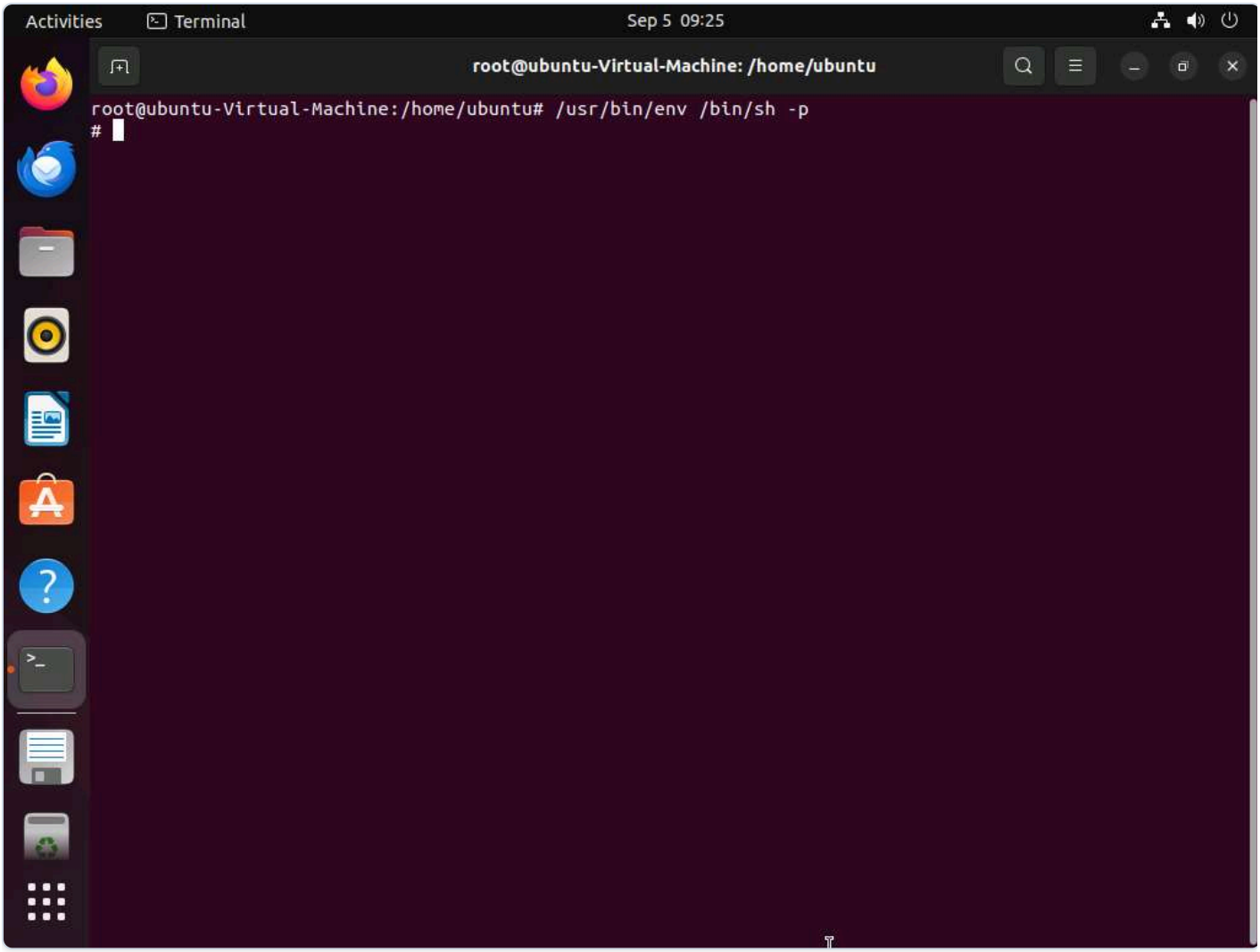
1. Scroll down to review the **Interesting Stuff** section displaying flagged SUID binaries.



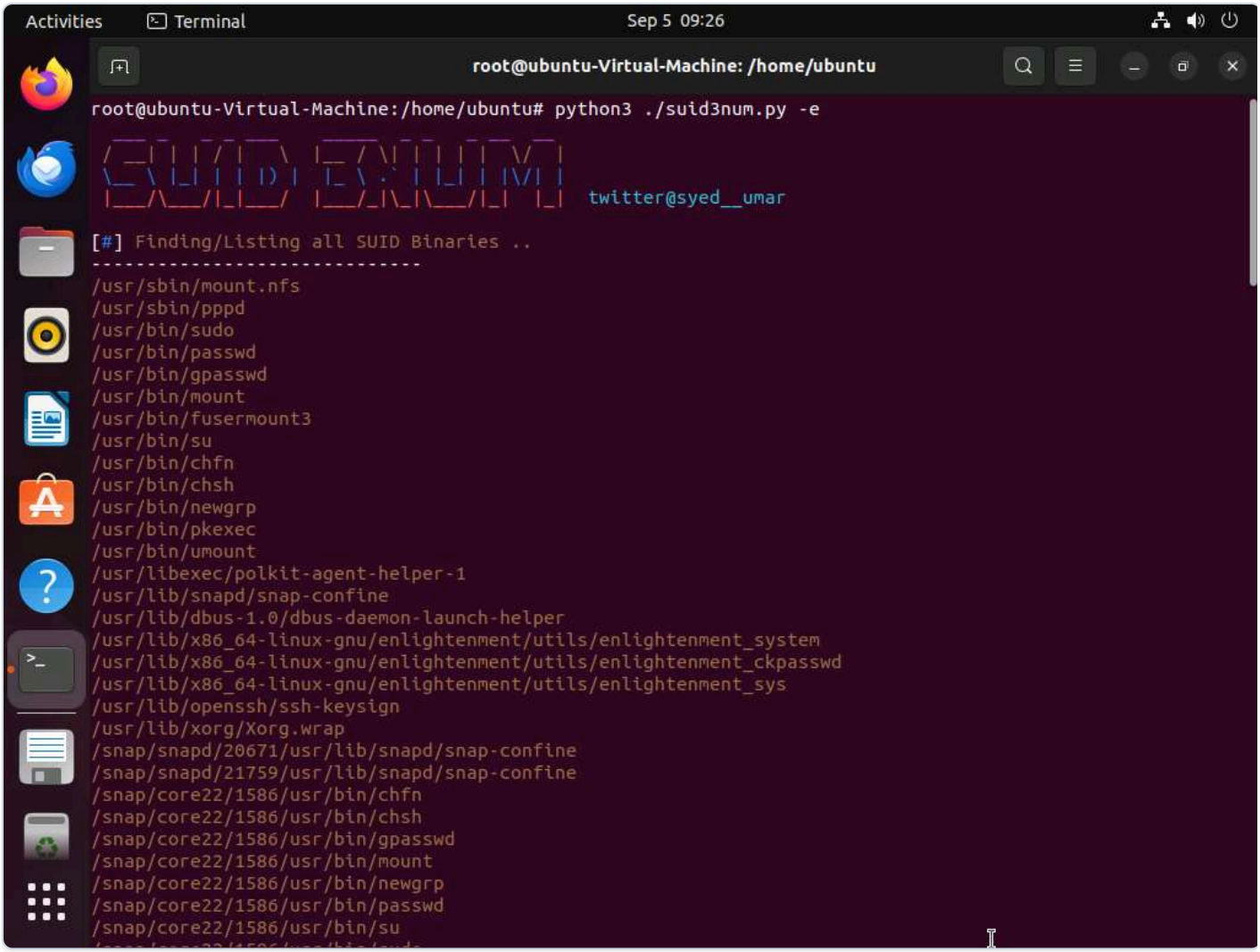
1. Continue scrolling to inspect the **GTFOBins list** detailing exploitable binaries and command syntax.



1. Review additional available binaries for shell breakout experiments.



1. Test the auto-exploit feature and observe execution results.



This concludes the lab exercise.

IMPORTANT: ❓ **Question 10.5.1.1** On Ubuntu machine, utilize tools such as LinPEAS and SUID3NUM to identify SUID binaries and assess their potential for privilege escalation. Identify the two suspicious files with the SUID bit set and list them, separated by a comma. [!SUCCESS] ✅ **Answer:** **/usr/bin/cp,/usr/bin/nano**